TRANSPORT AND TELECOMMUNICATION INSTITUTE
Faculty of Computer Science and Electronics

COURSE WORK

Object-Oriented Programming

Topic: «EventPlanner — Desktop Event Management System»

Student: Nikita Krokins
Group: 4301-2BDA
Student code: st81943

TABLE OF CONTENTS

Introduction

This course-work report describes the design and development of EventPlanner, a single-user desktop application for managing events such as conferences, trainings and workshops. The task was to build a complete object-oriented application that covers a real subject area end to end: planning events, managing the people who take part in them, registering participants with payment tracking, checking attendees in during the event, and reporting on the resulting finances.

The subject area — event organisation — is a good fit for object-oriented programming because it naturally contains distinct kinds of objects (events, people, registrations, expenses) with clear attributes and relationships, and because several of those objects share behaviour that can be expressed through inheritance and polymorphism. The application is written in C# on .NET 8, uses Windows Presentation Foundation (WPF) for the user interface, and stores its data in a local SQLite database.

The report is organised as follows. Chapter 1 lists the tasks of the course project. Chapter 2 analyses the subject area, its core entities, their attributes and relationships, and the fields used for searching and sorting. Chapter 3 presents the design: navigation, user-defined types, data storage, the class diagram and algorithm flowcharts. Chapter 4 describes the development — the implemented types, the user interface, the list of source files, the GitHub repository and screenshots of the running application. Chapter 5 covers testing, including the test scenarios, results, and the defects that were found and corrected. Chapter 6 gives the conclusions, and Chapter 7 lists the references. The full source code is provided as an annex.

1. Tasks for the Course project

The aim of the course project is to design and implement an object-oriented desktop application for a chosen subject area, and to demonstrate the core principles of object-oriented programming together with a clean, layered software architecture. The concrete tasks were defined as follows.

1.1 Functional tasks

- Manage events with full create, read, update and delete (CRUD) operations and validation.
- Manage two kinds of people — attendees and speakers — including deactivating and reactivating them.
- Register participants for events, enforcing event capacity and preventing duplicate registrations.
- Support free events and zero-amount payments alongside paid registrations.
- Track each registration's payment status across five states: Free, Unpaid, PartPaid, Paid and Cancelled.
- Record per-event expenses with a category, amount and paid date.
- Show live financial summaries for an event: expected income, paid income, outstanding amount, total expenses and profit.
- Provide a check-in screen that records attendance during the event with a single click.
- Export three CSV reports: the full attendee list, the list of unpaid registrations and a finance summary.

1.2 Non-functional tasks

- Demonstrate object-oriented design: inheritance, polymorphism, encapsulation, abstraction and interfaces.
- Use a layered architecture with the Repository pattern and an MVVM presentation layer.
- Persist all data in a file-based database with referential integrity.
- Validate input at the domain, service and database layers.
- Keep the business logic free of UI dependencies so that it can be unit-tested.
- Protect the application against common data-handling problems (SQL injection and CSV formula injection).

2. Analysis of the subject area

The subject area is the organisation of events. An event organiser needs to keep track of which events are planned, who is taking part in each event, whether participants have paid, who actually attended, and whether the event made a profit once expenses are taken into account. These needs map directly onto a small number of core objects.

2.1 Core entities and their attributes

Four core entities were identified. People are modelled as an abstract base type with two concrete subtypes, because attendees and speakers share most attributes but differ in a few. The tables below list the main attributes of each entity.

Event

Attribute	Type	Description
EventId	int	Primary key, generated by the database.
Title	string	Event name (required).
StartAt	DateTime	Date and time the event starts.
Location	string	Where the event takes place.
Capacity	int	Maximum number of active registrations.
Status	EventStatus	Draft, Open, Closed or Archived.
Description	string	Optional free-text description.

Person (abstract), Attendee, Speaker

Attribute	Type	Description
PersonId	int	Primary key.
FullName	string	Person's full name (required).
Phone	string	Contact phone (required).
Email	string	Optional e-mail address.
Role	PersonRole	Attendee or Speaker (set by the subtype).
IsActive	bool	Whether the person is currently active.
Notes	string	Attendee only — free-text notes.
Topic / Bio	string	Speaker only — talk topic and biography.

Registration

Attribute	Type	Description
RegistrationId	int	Primary key.
EventId / PersonId	int	Foreign keys to Event and Person.
TicketType	string	Ticket label, e.g. Standard or Speaker.
Price / PaidAmount	decimal	Ticket price and the amount paid so far.
Status	PaymentStatus	Free, Unpaid, PartPaid, Paid or Cancelled.
CheckedInAt	DateTime?	Time of check-in, or empty if not checked in.
CreatedAt	DateTime	When the registration was created.

Expense

Attribute	Type	Description
ExpenseId	int	Primary key.
EventId	int	Foreign key to the owning event.
Category	string	Expense category, e.g. Room rent, Catering.
Amount	decimal	Cost of the expense.
PaidAt	DateTime?	When the expense was paid, if at all.

2.2 Relationships between objects

The entities are related as follows: one Event has many Registrations and many Expenses; one Person (attendee or speaker) can have many Registrations. A Registration therefore links exactly one Event with exactly one Person. To prevent the same person being registered twice for the same event, the pair (EventId, PersonId) is unique. Deleting an event cascades to its registrations and expenses, so the data can never be left with orphaned rows.

2.3 Searching and sorting

The attributes used for searching and sorting were chosen from how an organiser actually works with the data:

- Events are searched by title, location or status, because those are the fields an organiser remembers an event by. The events list also shows summary cards so the most useful aggregate numbers are visible without sorting.
- People are searched by name, phone or e-mail — the three ways a participant is usually identified.
- On the check-in screen, registrations are filtered by typing part of the participant's name, which is the fastest way to find someone at the door.
- Registrations are listed newest-first (sorted by CreatedAt descending) so the most recent activity is at the top.
- Financial figures are aggregated (summed) across the registrations and expenses of an event rather than sorted, because the organiser cares about totals, not order.

3. Design

3.1 Navigation and menu design

The application uses a single main window with a fixed sidebar on the left and a content area on the right. The sidebar contains the application logo and six navigation buttons. Selecting a button swaps the content area to the corresponding page; double-clicking an event opens a dedicated event-details page. This keeps the whole application inside one window with no nested menus, which suits a single-user desktop tool.

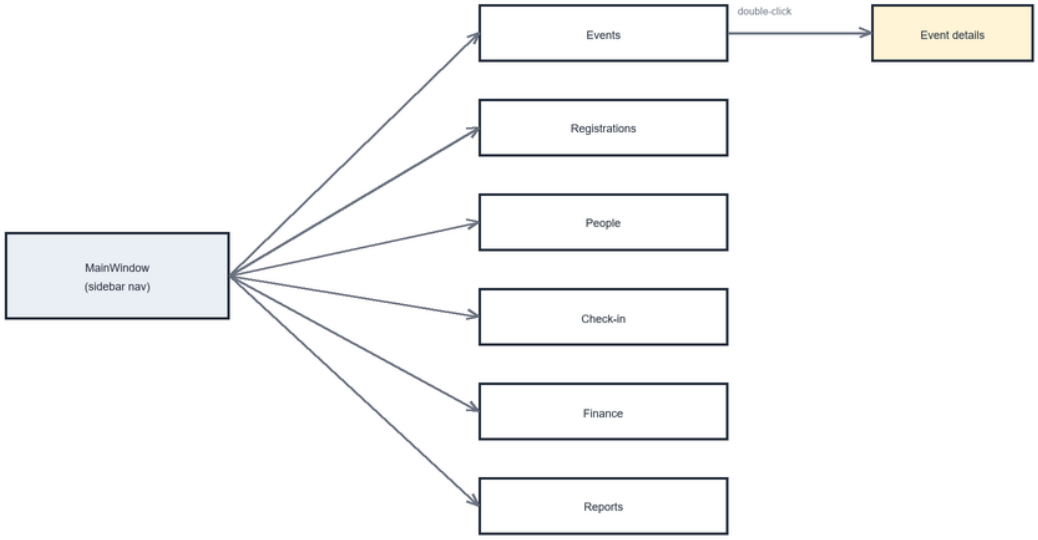

Figure 1. Navigation map — sidebar buttons and the six pages, plus the event-details page

3.2 User-defined types

People are modelled with an abstract base class Person and two concrete subclasses, Attendee and Speaker. This is the clearest example of inheritance in the project: both subclasses share the common attributes (name, phone, e-mail, role, active flag) while each adds its own (notes for attendees; topic and biography for speakers). The Speaker subclass also overrides validation to require a topic. The Event, Registration and Expense entities are plain classes. All entities implement a common IValidatable interface, and all four repositories implement a generic IRepository<T> interface.

3.3 Data storage

Data is stored in a local SQLite database file, eventplanner.db, created automatically next to the executable on first run. SQLite was chosen because it is file-based, requires no server, and is ideal for a single-user application. Four tables hold the data, with foreign keys enabled per connection (PRAGMA foreign_keys = ON) and ON DELETE CASCADE on the registration and expense foreign keys.

```
Events       (EventId PK, Title, StartAt, Location, Capacity, Status, Description)
People       (PersonId PK, FullName, Phone, Email, Role, IsActive, Notes, Topic, Bio)
Registrations(RegistrationId PK, EventId FK, PersonId FK, TicketType, Price,
              PaidAmount, Status, Notes, CheckedInAt, CreatedAt,
              UNIQUE(EventId, PersonId))
Expenses     (ExpenseId PK, EventId FK, Category, Amount, Notes, PaidAt, CreatedAt)
```

3.4 Class diagram

The class diagram below shows the domain model: the Person hierarchy, the Event, Registration and Expense classes, and the associations between them with their multiplicities.

Figure 2. Domain class diagram (Person hierarchy and the Event–Registration–Expense relationships)

3.5 Software architecture

The application is organised into layers, where each layer depends only on the layers below it. The repositories are the only place that communicates with SQLite; every layer above works with plain domain objects.

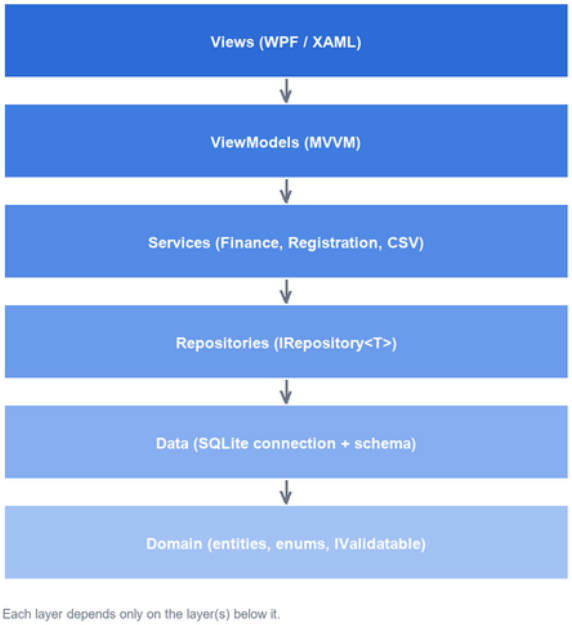

Figure 3. Layered architecture of the application

3.6 Algorithms (flowcharts)

Two of the most important algorithms are the derivation of a registration's payment status and the creation of a registration. Their flowcharts are shown below.

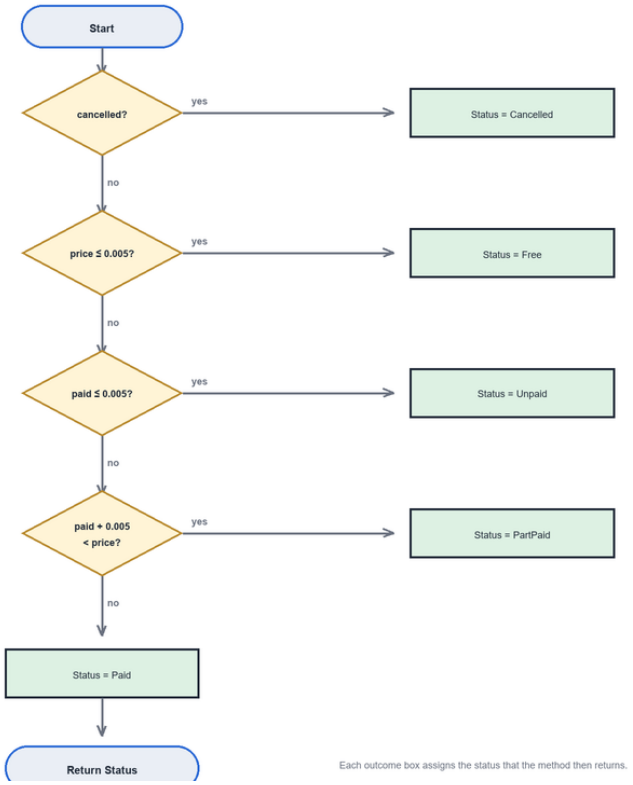

Figure 4. Flowchart of Registration.ComputeStatus — deriving payment status from price and paid amount

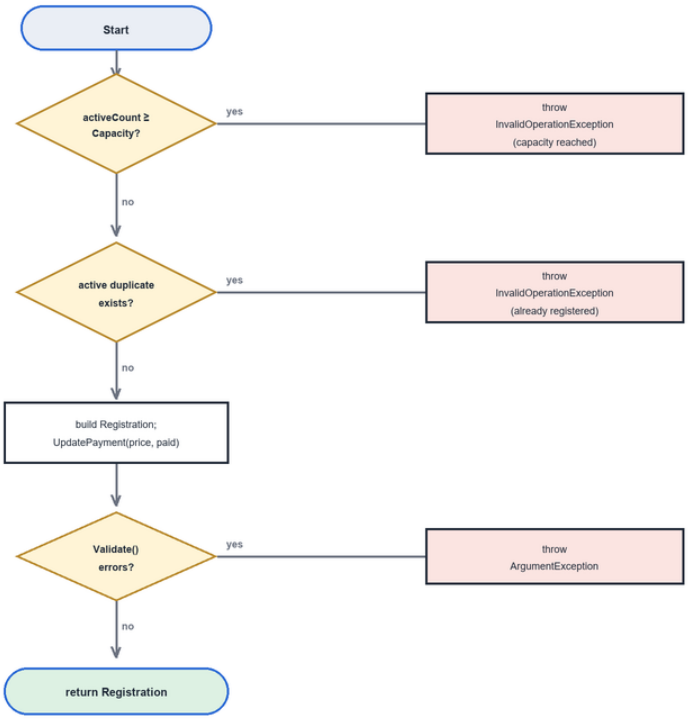

Figure 5. Flowchart of RegistrationService.CreateRegistration — capacity and duplicate checks before persistence

4. Development

The application was developed in C# 12 on .NET 8 using Visual Studio 2022, with Microsoft.Data.Sqlite 8.0.5 for data access and NUnit 3 for unit testing. This chapter describes the user-defined types, the user interface, the list of source files, the source-code repository and the running application.

4.1 Technology stack

Component	Choice
Language	C# 12
Framework	.NET 8 (net8.0-windows)
User interface	WPF with XAML
Architecture	MVVM with the Repository pattern
Database	SQLite (file-based, single-user)
Data access	Microsoft.Data.Sqlite 8.0.5
Testing	NUnit 3 with TestCase parameterisation

4.2 Key user-defined types

The Registration class is the richest type. It encapsulates its money and status fields with private setters so that the object can only be changed through methods that keep it consistent.

Member	Kind	Description
Price, PaidAmount	property (private set)	Money values; can only be changed via UpdatePayment().
Status	property (private set)	Payment status; derived, never set directly from outside.
UpdatePayment(price, paid, cancelled)	method	Sets price and paid amount and recomputes the status.
Cancel()	method	Marks the registration as Cancelled.
CheckIn()	method	Stamps the check-in time; refuses cancelled registrations.
ComputeStatus(price, paid, cancelled)	static method	Pure function returning the PaymentStatus for given values.
Validate()	method	Returns a list of validation error messages.

The Person hierarchy demonstrates inheritance and polymorphism: Speaker overrides Validate() to add a topic requirement, and PersonRepository returns the correct subclass (Attendee or Speaker) based on the Role column. Business rules that cross several entities live in services: RegistrationService (capacity and duplicate checks), FinanceService (income, expense and profit aggregation) and CsvExportService (the three RFC 4180 exports).

4.3 User interface

The interface consists of one main window, six pages and four dialog windows. The table summarises each page.

Page	Behaviour
Events	Create, edit, archive and delete events; search by title, location or status; double-click a row to open details; summary cards for income and event count.
Event details	Per-event view with Info, Registrations, Speakers and Finance tabs, and a direct Add Registration button.
People	CRUD for attendees and speakers; Deactivate / Reactivate toggle; search by name, phone or e-mail; the form switches fields by role.
Registrations	CRUD with capacity and duplicate checks; a free-entry toggle that zeroes price and paid amount; inactive people are hidden when creating new registrations.
Check-in	A filterable list of active registrations; one click records the check-in time; already-checked-in rows are marked.
Finance	Five summary cards (Expected, Paid, Outstanding, Expenses, Profit) and CRUD for per-event expenses.
Reports	Three CSV exports with RFC 4180 quoting: attendee list, unpaid list and finance summary.

4.4 List of source files

Folder / file	Responsibility
Domain/	Entities (Event, Person, Attendee, Speaker, Registration, Expense), enums and the IValidatable interface.
Data/Database.cs	SQLite connection, schema creation and foreign-key pragma.
Repositories/	EventRepository, PersonRepository, RegistrationRepository, ExpenseRepository — all implementing IRepository<T>.
Services/	FinanceService, RegistrationService, CsvExportService.
ViewModels/	MainViewModel, EventRowViewModel, RegistrationRowViewModel, ViewModelBase.
Commands/RelayCommand.cs	ICommand implementation for sidebar navigation.
Views/	MainWindow, six page UserControls and four dialog windows (XAML + code-behind).

4.5 Source-code repository (GitHub)

The complete source code is kept under Git version control. Repository URL:

Https://github.com/nkvoronovich-cyber/eventplanner

A local Git repository has already been initialised with a clean history and a .gitignore that excludes build output and the database file. To publish it, create an empty repository on GitHub and run:

The complete source code of the EventPlanner application is stored in a public GitHub repository.

The repository contains the source code of the WPF application, project documentation, the final report, test scenarios, and the executable build folder. The repository is public and will remain available for at least one year.

To run the project from source code, the user should open the Source/EventPlanner.sln file in Visual Studio 2022, restore NuGet packages if required, build the solution, and run the application. The SQLite database file is created automatically on the first start of the application.

```
Repository URL:
https://github.com/nkvoronovich-cyber/eventplanner
```

4.6 Screenshots of the running application

The following screenshots demonstrate the developed EventPlanner application and show the main windows used by the user. The screenshots include the event management page, people management page, registration section, finance section, check-in section, and examples of application output.

These screenshots were captured from the running WPF application and demonstrate the visual interface, navigation menu, data tables, and main user actions.

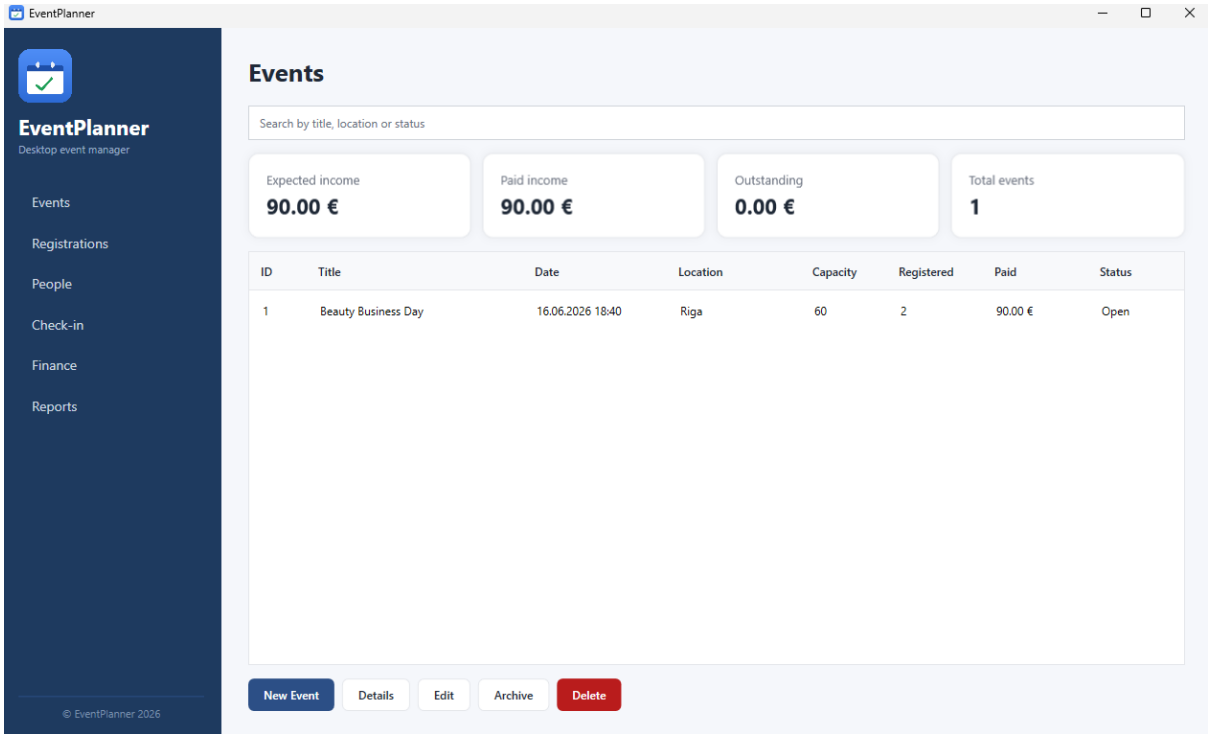

Figure 6. The Events page with the sidebar, search and summary cards

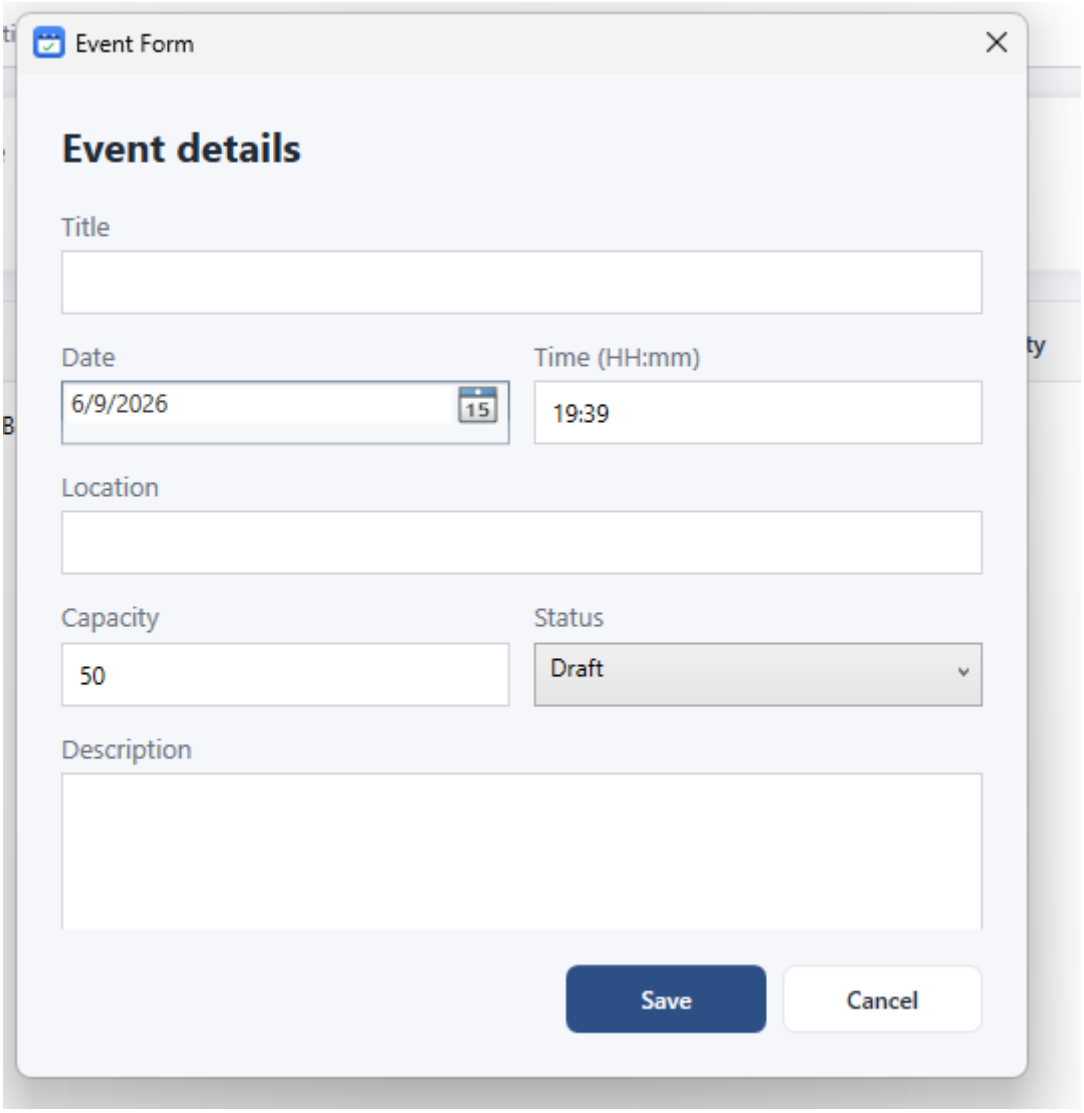

Figure 7. The Event-details page

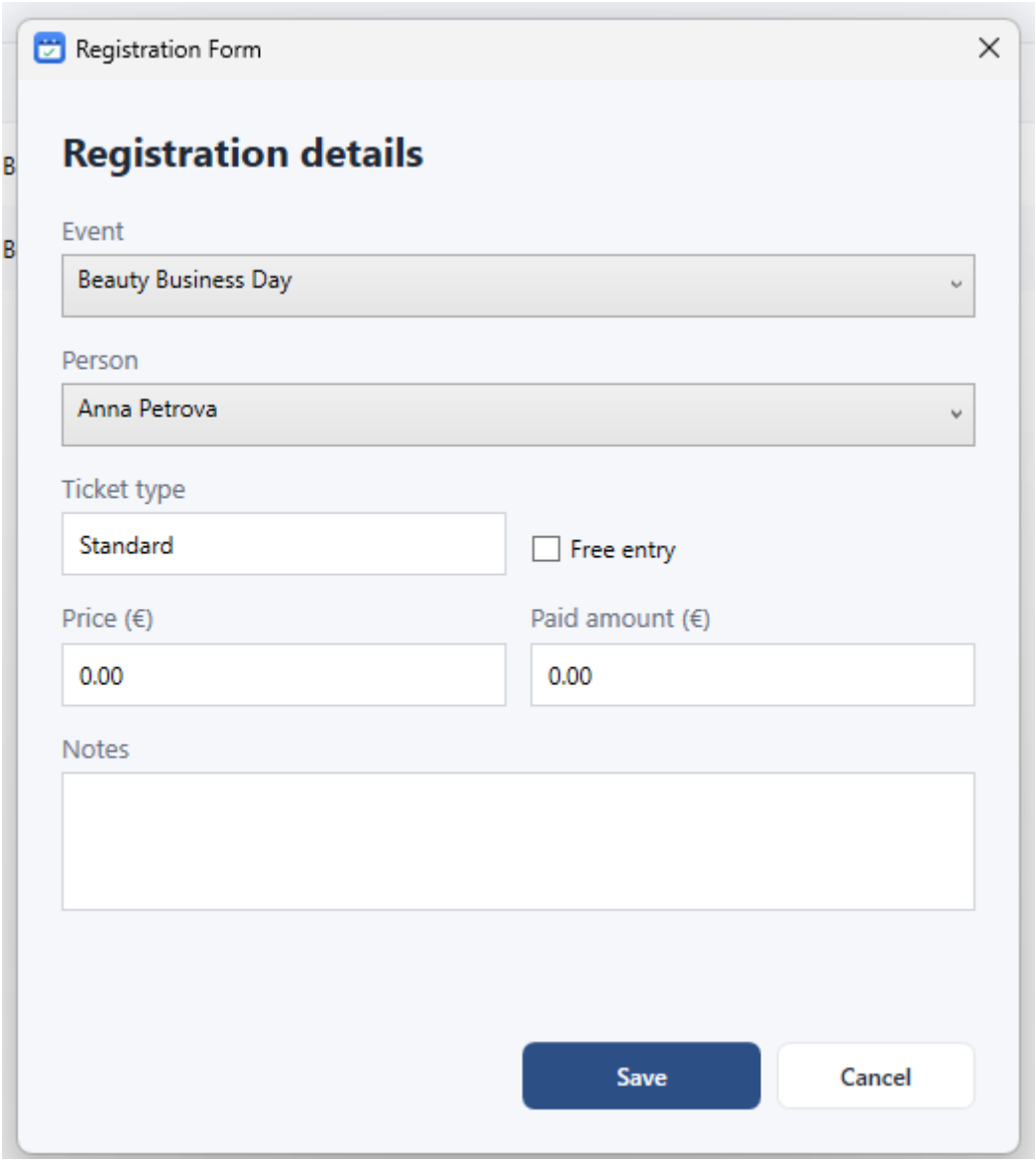

Figure 8. The Add / edit registration dialog with the free-entry toggle

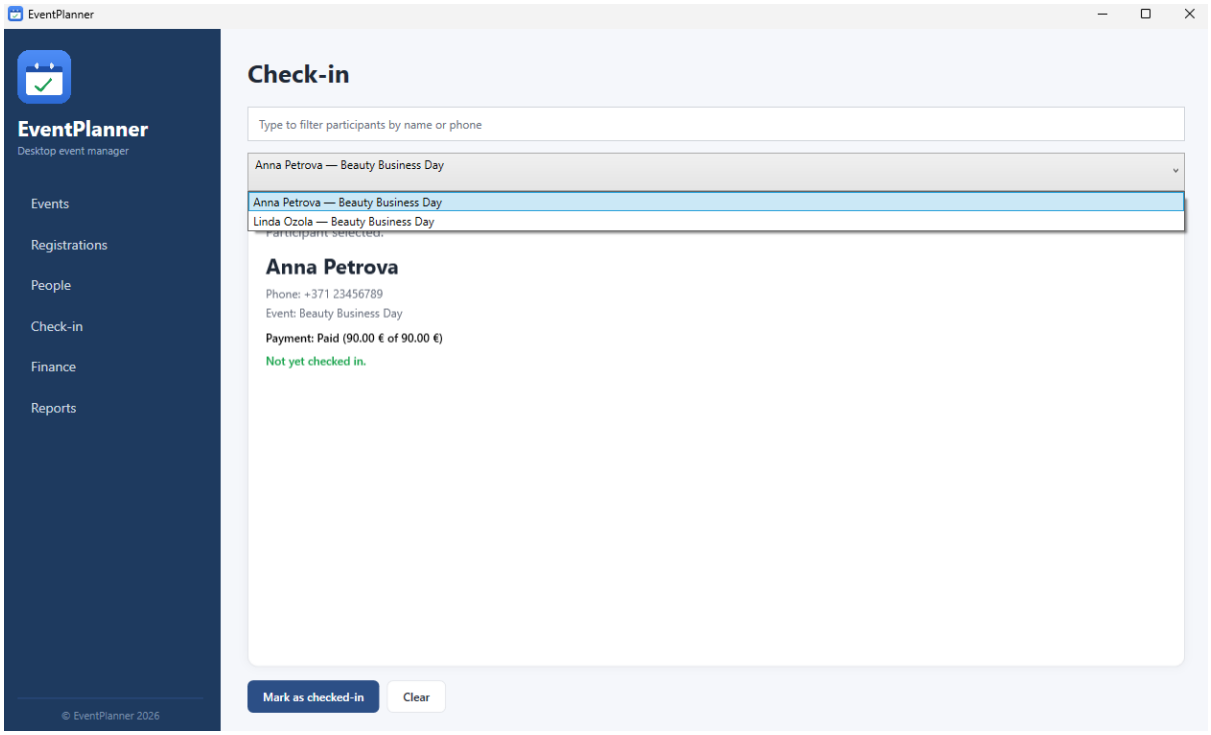

Figure 9. The Check-in page used during the event

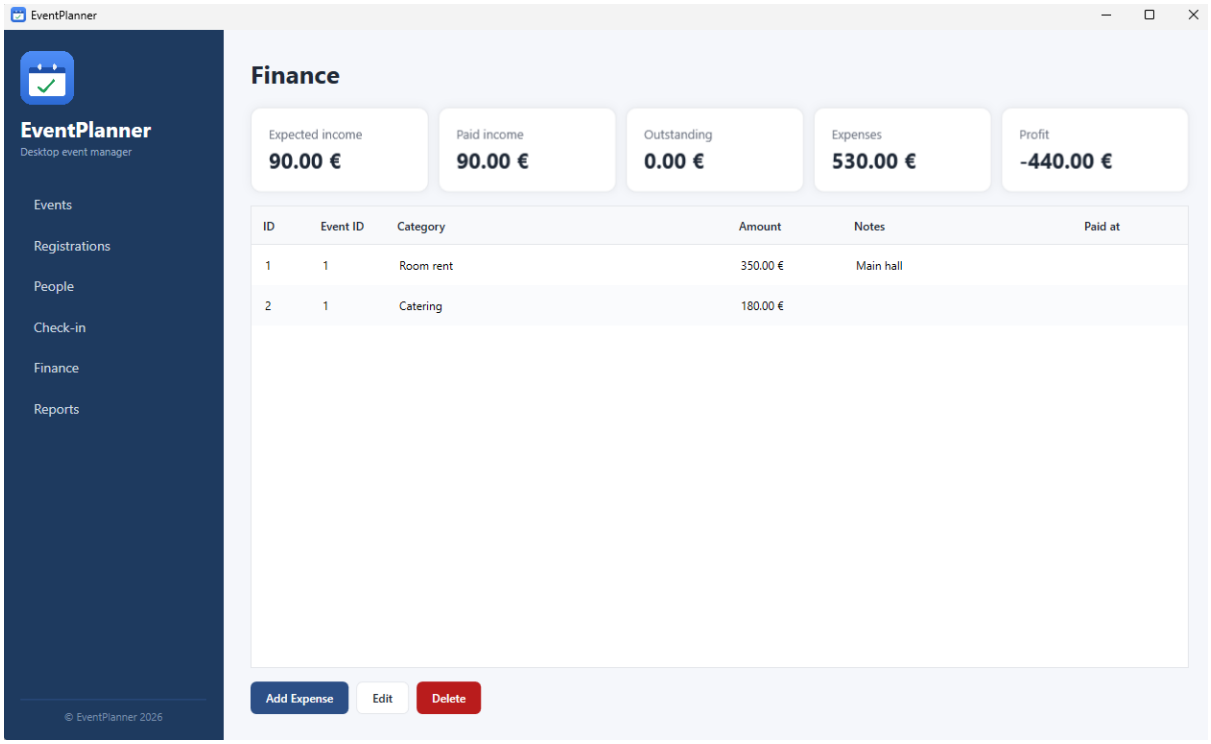

Figure 10. The Finance page with income, expenses and profit

RegistrationId	EventId	PersonId	TicketType	Price	PaidAmount	Status	CheckedIn
1	1	1	Standard	90	90	Paid	
2	1	2	Speaker	0	0	Free	

Figure 11. The Reports page and an exported CSV opened in a spreadsheet

5. Testing

5.1 How testing was organised

Testing was organised on two levels. First, validation is enforced at three layers of the application: the domain entities validate themselves through Validate(); the RegistrationService enforces cross-entity rules (capacity and no duplicate registration); and the database enforces UNIQUE, NOT NULL and foreign-key constraints as a last line of defence. Second, the business logic — which has no UI dependencies — is covered by automated unit tests written with NUnit.

5.2 Functions chosen for unit testing

- Registration.ComputeStatus — a pure function that derives the payment status from price and paid amount, including a 0.005 tolerance to absorb floating-point round-tripping.
- RegistrationService.CreateRegistration — enforces the capacity and duplicate-registration business rules.
- FinanceService.Profit — computes paid income minus total expenses, excluding cancelled registrations.

A complete NUnit test class with 13 parameterised cases is provided for ComputeStatus, and full test-scenario tables for all three functions (plus the two methods changed during the review) are given in the accompanying document EventPlanner_Test_Scenarios.docx.

5.3 Errors found and how they were corrected

A correctness and security review of the working application found two defects, both of which were corrected.

5.3.1 Re-registration after cancellation

Symptom: registering a participant, cancelling that registration, and registering the same participant for the same event again failed with a raw database error (“UNIQUE constraint failed”) shown to the user.

Cause: the no-duplicate rule in the service only counted non-cancelled registrations, so it allowed the action, but the database UNIQUE(EventId, PersonId) constraint applies to every row, including the cancelled one.

Correction: a new RegistrationRepository.AddOrReactivate method reuses and reactivates the existing cancelled row instead of inserting a duplicate. The user no longer sees an error and no duplicate rows are created.

5.3.2 CSV formula injection in exports

Symptom and risk: a text field beginning with =, +, - or @ is interpreted as a formula when an exported CSV is opened in a spreadsheet, which could execute unwanted content (CSV formula injection).

Correction: the shared Escape routine in CsvExportService now prefixes any such value with an apostrophe so the spreadsheet treats the cell as literal text, with the RFC 4180 quoting still applied on top. Because all three exports share this routine, every column of every report is protected.

6. Conclusions

Within the course project, a complete object-oriented desktop application was designed and implemented. EventPlanner covers the full lifecycle of an event: planning, managing attendees and speakers, registering participants with capacity and duplicate checks, tracking payments across five states, checking attendees in, recording expenses, summarising the finances, and exporting three CSV reports.

The application demonstrates the object-oriented principles it set out to: inheritance and polymorphism through the Person / Attendee / Speaker hierarchy, encapsulation through the Registration class's private setters and controlled mutators, abstraction through the IValidatable and IRepository<T> interfaces, and a clean layered architecture with the Repository pattern and an MVVM presentation layer. All data is stored with referential integrity in SQLite, input is validated at three layers, and the code is parameterised against SQL injection.

The main challenges during development were keeping the business logic free of UI dependencies so that it could be unit-tested, and getting the agreement between the service-level and database-level duplicate rules right — the latter surfaced as the re-registration defect that was found and fixed during the review. The parts that went well were the layered design, which made the defects easy to locate and correct, and the validation strategy, which catches bad input early.

Possible improvements include storing money as integer cents rather than floating-point values for exact arithmetic, adding more automated tests around the services and repositories, and extending the application towards multi-user scenarios. Overall, the project meets the functional and non-functional tasks defined in Chapter 1.

7. References

1. Microsoft. .NET 8 documentation. https://learn.microsoft.com/dotnet/
2. Microsoft. Windows Presentation Foundation (WPF) documentation. https://learn.microsoft.com/dotnet/desktop/wpf/
3. Microsoft. Microsoft.Data.Sqlite documentation. https://learn.microsoft.com/dotnet/standard/data/sqlite/
4. SQLite. SQLite documentation. https://www.sqlite.org/docs.html
5. Microsoft. Model-View-ViewModel (MVVM) pattern. https://learn.microsoft.com/dotnet/architecture/maui/mvvm
6. NUnit. NUnit 3 documentation. https://docs.nunit.org/
7. Y. Shafranovich. Common Format and MIME Type for CSV Files (RFC 4180). https://www.rfc-editor.org/rfc/rfc4180
8. OWASP. CSV Injection. https://owasp.org/www-community/attacks/CSV_Injection

Annex A. Source code

The full source code is provided in the accompanying archive (Source folder with the EventPlanner.sln solution) and in the Git repository linked in section 4.5. The test scenarios and the NUnit test class are provided in EventPlanner_Test_Scenarios.docx.